Evaluating Test Completeness Against Software Requirements – Formalism

Bc. Jakub Bláha, Bc. Jan Klanica, Bc. Pavel Lukl, Bc. Timea Adamčíková

June 6, 2026

Contents

1 Introduction

This document defines a logic-agnostic formal base for expressing software requirements and test cases. It is intended as a close-to-natural-language intermediate representation that an LLM (or a human) can read and produce: its constructs, expressions, and predicates are named after the vocabulary already used when describing requirements informally, while each name is tied to a precise mathematical definition. Its purpose is to enable mechanical detection of coverage gaps: given a set of requirements and a set of test cases formalised over a shared set of entities, a checker determines which requirement conditions are not exercised by any test.

The base is organised into several layers, each building on the ones below it. At the bottom, a time set and a set of *entities* (signals, storage, channels, states, events, and so on) describe the kinds of things present in the system, and a *trace* assigns to each time point a valuation mapping entities to their current values. Built on top of traces, *expressions* compute values and *predicates* evaluate to truth values at a given point in time. *Operations* are the actions a test case fires to drive a trace, each producing an effect predicate that states what must hold as a result of the firing. *Temporal constructs* then lift point-in-time predicates into statements about the whole trace by quantifying over time.

A requirement and a test case are both built from these same layers over the same entities, drawing their predicates from the same space; this shared vocabulary is what allows a checker to determine which conditions of a requirement a given test case exercises. The catalogues of entity kinds and operations are kept open to extension, so that supporting a new kind of entity or action requires only widening the relevant catalogue while every existing construct continues to work unchanged.

2 The primitives

These primitive sets serve as the base that the rest of the formalism builds on. The primitive sets are the following:

- T_D, T_C – the *discrete time* and *continuous time* sets respectively.
- ID_E – the set of entity identifiers.
- V_{base} – the base values set: booleans, reals, time points, and intervals.
- V_{nonset}, V – defined in Section 3 once entities are available; V_{nonset} is the union of base values and entities; V additionally includes finite sets whose elements are drawn from V_{nonset}.
- *EventTypeLabel* – the set of event type labels; defined below.

Definition 1 (Discrete time set)**.** *The* discrete time set T_D *is totally ordered and countable, extended with a greatest element* ∞*:*

$$T_D = \mathbb{N} \cup \{\infty\}$$

The ordering is the usual order on $\mathbb{N}$ *with* $\forall n \in \mathbb{N} : n < \infty$*.*

Definition 2 (Continuous time set)**.** *The* continuous time set T_C *is totally ordered and uncountable, extended with a greatest element* ∞*:*

$$T_C = \mathbb{R}_{\geq 0} \cup \{\infty\}$$

The ordering is the usual order on $\mathbb{R}_{\geq 0}$ *with* $\forall x \in \mathbb{R}_{\geq 0} : x < \infty$*.*

We say that T is an infinite totally ordered set; the time set can have *discrete* or *continuous* flavour; In the scope of a single requirement, T is equal either to T_D or T_C.

Definition 3 (Duration)**.** *A* duration *is a non-negative element of the time set, representing the length of a time span:*

$$\Delta = T$$

Thus $\Delta = \mathbb{N}$ *for discrete time and* $\Delta = \mathbb{R}_{\geq 0}$ *for continuous time.*

Definition 4 (Interval)**.** *An* interval *is a tuple of a start time point, a duration, and two endpoint-openness flags:*

$$\mathcal{I} = T \times \Delta \times \mathbb{B} \times \mathbb{B}$$

An interval $(t, d, \ell, r) \in \mathcal{I}$ *represents a time span with start* t*, end* $t + d$*, left-open flag* ℓ*, and right-open flag* r*:*

ℓ	r	*represented span*
false	*false*	$[t,\ t+d]$
true	*false*	$(t,\ t+d]$
false	*true*	$[t,\ t+d)$
true	*true*	$(t,\ t+d)$

Definition 5 (Entity identifier set)**.** *The* entity identifier set ID_E *is a finite nonempty set of unique abstract tokens called* entity identifiers*.*

Definition 6 (Base values)**.** *The* base values set V_{base} *is:*

$$V_{base} = \mathbb{B} \cup \mathbb{R} \cup T \cup \mathcal{I}$$

where $\mathbb{B} = \{true, false\}$ *is the set of booleans,* $\mathbb{R}$ *is the set of real numbers,* T *is the time set, and* $\mathcal{I}$ *is the set of intervals.*

Definition 7 (Event type label set)**.** *The* event type label set *EventTypeLabel is a finite set of abstract symbols:*

$$\begin{aligned} EventTypeLabel = \{\ &generic,\ calculated,\ written,\ read,\ entered,\ transmitted,\\ &received,\ inserted,\ removed,\ cleared\ \} \end{aligned}$$

3 Entities and entity types

An entity type is an item belonging to a set of entity types. There is a number of defined entity types and this set is open to extension.

Definition 8 (Entity type set)**.**

$$T_E = \{Signal, Storage, Event, EventTrigger, Channel, State, Value, Abstract, Set\}$$

Entity type modifiers are properties which can be attached to an entity of a particular type. A modifier is either a *flag*, whose presence alone carries meaning, or a *parameterised modifier*, which additionally carries a value in $ModVal$.

Definition 9 (Modifier value set)**.** *The* modifier value set *is:*

$$ModVal = V_{base} \cup ID_E \cup EventTypeLabel$$

Each entity type is compatible with only a subset of the modifier keys. Therefore we define the *entity type modifiers function.*

Definition 10 (Entity type modifiers function)**.** $M_E : T_E \to \mathcal{P}(ModKey)$ *maps each entity type to its set of permitted modifier keys, where ModKey is a finite nonempty set of modifier keys. The permitted modifier keys and their expected value types are:*

Entity type	***Modifier key***	***Value type***
Storage	*address*	$\mathbb{N}$
Event	*target*	ID_E
Event	*type*	*EventTypeLabel*

Every `Event` *entity is expected to carry both modifiers: user-declared events set target to the identifier of their* `EventTrigger` *and type to generic; predefined events set target to their parent entity and type to the appropriate event type.*

Note: At most one `Event` *entity with a given* $(target, type)$ *pair should exist in* E *for a given requirement.*

An entity is something that is referenced in a requirement and corresponding test cases. Formally, an entity is defined as a three-item tuple.

Definition 11 (Entity)**.**

$$e = (id_e, t_e, \mu_e)$$

, where

- id_e *is the unique identifier of the entity;* $id_e \in ID_E$
- t_e *is the type of the entity;* $t_e \in T_E$
- μ_e *is the modifier assignment function of the entity;* $\mu_e : ModKey \rightharpoonup ModVal$, $\mathrm{dom}(\mu_e) \subseteq M_E(t_e)$ [1]

No two entities in E *share the same identifier:* $\forall e, e' \in E : e \neq e' \Rightarrow id_e \neq id_{e'}$*. We write* E *for the set of all entities. The set* E *is finite and nonempty.*[2]

Definition 12 (Non-set values)**.** *The* non-set values V_{nonset} *is the union of base values and entities:*

$$V_{nonset} = V_{base} \cup E$$

Definition 13 (Values)**.** *The* values set V *is:*

$$V = V_{nonset} \cup \mathcal{P}_{fin}(V_{nonset})$$

where $\mathcal{P}_{fin}(V_{nonset})$ *is the set of all finite subsets of* V_{nonset}*. Set values are flat: they may contain only non-set values.*

4 Valuations and traces

Definition 14 (Valuation)**.** *A valuation is a partial function* $\sigma : E \rightharpoonup V$ *assigning a value to a subset of entities at each time point*[3]*;* $\sigma(e)$ *is undefined when the entity has no defined value at that time point. For entities of type* `Event`*,* $\sigma(e)$ *is always defined and* $\sigma(e) \in \{true, false\}$*, where* $\sigma(e) = true$ *iff the event occurs at that time point. For entities of type* `Set`*,* $\sigma(e)$*, when defined, is a finite subset of* V_{nonset}*, i.e.* $\sigma(e) \in \mathcal{P}_{fin}(V_{nonset})$*.*

We write Σ for the infinite set of all valuations.

Definition 15 (Trace)**.** *A trace is a function* $\rho : T \to \Sigma$*. We write* $\rho(t)(e)$ *for the value of entity* e *at time* t*.*

We write Σ^T for the set of all traces, i.e. the set of all functions from T to Σ.
[4]

5 Expressions and predicates

All expression functions defined in this section evaluate to some value $v \in V$. We write **Expr** for the infinite set of all expressions.

[1] $\mathrm{dom}(\mu_e)$ is a subset of the set of modifier keys compatible with the entity type t_e. It is not required that a value is defined for each modifier key compatible with t_e.

[2] The set E is finite in the scope of a single requirement.

[3] For instance a register entity can have its value undefined at time points preceeding a write to this register.

[4] This is standard function space notation: $\Sigma^T = \{\, \rho \mid \rho : T \to \Sigma \,\}$.

5.1 Expressions

5.1.1 Constants

Definition 16 (Constant)**.** *Any value $c \in V$ may appear as a constant expression.*

5.1.2 Arithmetic operators

Definition 17 (Arithmetic operators)**.** *The standard arithmetic operators $+$, $-$, $\times$, $/$ are defined on numeric values ($\mathbb{R}$ and its subsets) in the usual sense and can be used to construct expressions.*

5.1.3 Time functions

The following functions depend only on time points or intervals; they do not depend on a trace.

Definition 18 (Current time)**.** *$Now \in T$ returns the ambient time point t:*

$$Now \;=\; t$$

The following two functions are defined for **discrete time only** ($T = T_D = \mathbb{N}$).

Definition 19. *$Prev : T_D \rightharpoonup T_D$ returns the preceding time step:*

$$Prev(t) \;=\; t - 1$$

The function is partial: it is undefined at $t = 0$ since $t - 1 \notin T_D$.

Definition 20. *$Next : T_D \to T_D$ returns the following time step:*

$$Next(t) \;=\; t + 1$$

Definition 21. *$RelTime : T_D \times \mathbb{Z} \rightharpoonup T_D$ shifts a time point by n steps, where $n \in \mathbb{Z}$ may be positive or negative:*

$$RelTime(t,\, n) \;=\; t + n$$

The function is partial: it is undefined when $t + n < 0$ since $t + n \notin T_D$. Note that $Prev(t) = RelTime(t, -1)$ and $Next(t) = RelTime(t, 1)$.

The following functions are defined for both time flavours.

Definition 22. *$Diff : T \times T \to \Delta$ returns the duration between two time points:*

$$Diff(t_1,\, t_2) \;=\; |t_1 - t_2|$$

Definition 23 (Interval constructors)**.** *Four constructors build intervals from a start and end time point, differing only in endpoint openness. All are partial: undefined when $t_1 > t_2$.*

$$\begin{aligned}
MkIntervalCC(t_1,\, t_2) &= (t_1,\; t_2 - t_1,\; false,\; false) && [t_1,\, t_2] \\
MkIntervalOC(t_1,\, t_2) &= (t_1,\; t_2 - t_1,\; true,\; false) && (t_1,\, t_2] \\
MkIntervalCO(t_1,\, t_2) &= (t_1,\; t_2 - t_1,\; false,\; true) && [t_1,\, t_2) \\
MkIntervalOO(t_1,\, t_2) &= (t_1,\; t_2 - t_1,\; true,\; true) && (t_1,\, t_2)
\end{aligned}$$

Definition 24 (Since-zero interval)**.** *$SinceZero : T \to \mathcal{I}$ constructs an interval from time zero to a given time point t:*

$$SinceZero(t) \;=\; (0,\; t)$$

Definition 25 (Interval start)**.** *$Start : \mathcal{I} \to T$ returns the start time point of an interval:*

$$Start((t,\, d,\, \ell,\, r)) \;=\; t$$

Definition 26 (Interval end)**.** *$End : \mathcal{I} \to T$ returns the end time point of an interval:*

$$End((t,\, d,\, \ell,\, r)) \;=\; t + d$$

Definition 27 (Interval duration)**.** *$Duration : \mathcal{I} \to \Delta$ returns the duration of an interval:*

$$Duration((t,\, d,\, \ell,\, r)) \;=\; d$$

Definition 28 (Interval endpoint openness)**.** *$IsLeftOpen$, $IsRightOpen : \mathcal{I} \to \mathbb{B}$ return the endpoint-openness flags:*

$$IsLeftOpen((t,\, d,\, \ell,\, r)) \;=\; \ell \qquad IsRightOpen((t,\, d,\, \ell,\, r)) \;=\; r$$

Definition 29 (Time point interval membership)**.** *$InInterval : T \times \mathcal{I} \to \mathbb{B}$ holds when time point s falls within interval i:*

$$\begin{aligned} InInterval(s,\, i) \;\Leftrightarrow\; & \big(IsLeftOpen(i) \Rightarrow Start(i) < s\big) \\ & \wedge \big(\neg IsLeftOpen(i) \Rightarrow Start(i) \le s\big) \\ & \wedge \big(IsRightOpen(i) \Rightarrow s < End(i)\big) \\ & \wedge \big(\neg IsRightOpen(i) \Rightarrow s \le End(i)\big) \end{aligned}$$

5.1.4 Entity attribute accessors

These expressions return static properties of entities and do not depend on the trace or time.

Definition 30 (Address)**.** *For a `Storage` entity $e \in E$ carrying an address modifier, $Addr : E \to V$ returns the static address value:*

$$Addr(e) \;=\; \mu_e(address)$$

5.1.5 Trace functions

The following functions additionally take a trace $\rho \in \Sigma^T$ and produce values that reflect the state of the system over time.

Definition 31. *For a trace $\rho \in \Sigma^T$, the partial function $Val_\rho : E \times T \rightharpoonup V$ denotes the value of entity $e \in E$ at time $t \in T$:*

$$Val_\rho(e,\, t) \;=\; \rho(t)(e)$$

The function is partial: $Val_\rho(e,\, t)$ is undefined when $e \notin \mathrm{dom}(\rho(t))$.

Definition 32 (Value before time point)**.** *For a trace $\rho \in \Sigma^T$, the partial function $ValBefore_\rho : E \times T \rightharpoonup V$ is the left limit of the value of e at t:*

$$ValBefore_\rho(e,\, t) \;=\; \lim_{t' \to t^-} Val_\rho(e,\, t')$$

The function is partial: it is undefined when the left limit does not exist, i.e. when no operation has fired on e before t.

Definition 33 (Event occurrence count)**.** *For a trace $\rho \in \Sigma^T$ and an entity $ev \in E$ of type `Event`, $EvtOccCount_\rho : E \times \mathcal{I} \to \mathbb{N}$ returns the number of occurrences of ev within an interval $i \in \mathcal{I}$:*

$$EvtOccCount_\rho(ev,\, i) \;=\; \big|\{\, s \in T \mid InInterval(s,\, i) \;\wedge\; Val_\rho(ev,\, s) = true \,\}\big|$$

Definition 34 (Last occurrences function)**.** *For a trace $\rho \in \Sigma^T$, an entity $ev \in E$ of type `Event`, a time point $t \in T$, and $n \in \mathbb{N}^+$, let $t_1 \le t_2 \le \cdots \le t_n$ be the n most recent occurrence times of ev up to t, i.e. the n largest elements of $\{\, s \in T \mid s \le t \wedge Val_\rho(ev,\, s) = true \,\}$ sorted in ascending order. Then $LastOcc_\rho : E \times T \times \mathbb{N}^+ \rightharpoonup \mathcal{I}$ is defined as:*

$$LastOcc_\rho(ev,\, t,\, n) \;=\; MkIntervalCC(t_1,\, t_n)$$

The function is partial: it is undefined when fewer than n occurrences of ev exist up to t.

Note: The time of a single occurrence is obtained via $LastOcc_\rho(ev, t, 1)$, which returns an interval of duration 0. The occurrence time is then $Start(LastOcc_\rho(ev, t, 1))$.

Definition 35 (First occurrences function)**.** *For a trace $\rho \in \Sigma^T$, an entity $ev \in E$ of type* `Event`*, a time point $t \in T$, and $n \in \mathbb{N}^+$, let $t_1 \leq t_2 \leq \cdots \leq t_n$ be the n earliest occurrence times of ev at or after t, i.e. the n smallest elements of $\{\, s \in T \mid s \geq t \ \wedge \ Val_\rho(ev, s) = true \,\}$ sorted in ascending order. Then $FirstOcc_\rho : E \times T \times \mathbb{N}^+ \rightharpoonup \mathcal{I}$ is defined as:*

$$FirstOcc_\rho(ev,\, t,\, n) \ = \ MkIntervalCC(t_1,\, t_n)$$

The function is partial: it is undefined when fewer than n occurrences of ev exist at or after t.

Note: The time of the first occurrence is $Start(FirstOcc_\rho(ev, t, 1))$.

Definition 36 (Maximum value function)**.** *For a trace $\rho \in \Sigma^T$ and an entity $e \in E$ with an ordered value domain, $MaxVal_\rho : E \times \mathcal{I} \to V$ returns the maximum value of e over an interval $i \in \mathcal{I}$:*

$$MaxVal_\rho(e,\, i) \ = \ \sup\{\, Val_\rho(e,\, s) \mid s \in T,\ Start(i) \leq s \leq End(i) \,\}$$

Definition 37 (Minimum value function)**.** *For a trace $\rho \in \Sigma^T$ and an entity $e \in E$ with an ordered value domain, $MinVal_\rho : E \times \mathcal{I} \to V$ returns the minimum value of e over an interval $i \in \mathcal{I}$:*

$$MinVal_\rho(e,\, i) \ = \ \inf\{\, Val_\rho(e,\, s) \mid s \in T,\ Start(i) \leq s \leq End(i) \,\}$$

5.1.6 Set functions

These functions operate on set values and do not depend on a trace.

Definition 38 (Set size)**.** *$Size : \mathcal{P}_{fin}(V_{nonset}) \to \mathbb{N}$ returns the cardinality of a set:*

$$Size(s) \ = \ |s|$$

Definition 39 (Set filter)**.** *For a set $s \in \mathcal{P}_{fin}(V_{nonset})$ and a predicate P parameterised by $x \in V_{nonset}$, $Filter : \mathcal{P}_{fin}(V_{nonset}) \times \mathbf{Pred} \to \mathcal{P}_{fin}(V_{nonset})$ returns the subset of elements satisfying P:*

$$Filter(s,\, P) \ = \ \{\, x \in s \mid P(x) \,\}$$

5.2 Predicates

A predicate is a function that evaluates to a value in $\mathbb{B}$. We write **Pred** for the infinite set of all predicates.

Definition 40 (Boolean constants)**.** $\top \in \mathbf{Pred}$ *and* $\bot \in \mathbf{Pred}$ *are the always-true and always-false predicates respectively.*

Definition 41 (Comparison predicate)**.** *For expressions $a, b \in V$ and a comparison operator $op \in \{=, \neq, <, \leq, >, \geq\}$:*

$$Cmp(a,\, op,\, b) \ = \ a \ op \ b$$

The operators $<, \leq, >, \geq$ require a, b to be from an ordered domain. Since $\Delta \subseteq \mathbb{R}$, durations are an ordered domain and duration comparison is a valid instance of Cmp.

Definition 42 (Set membership)**.** *For a value $v \in V_{nonset}$ and a set value $s \in \mathcal{P}_{fin}(V_{nonset})$:*

$$Contains(v,\, s) \ \Leftrightarrow \ v \in s$$

Definition 43 (Negation)**.** *For a predicate $P \in \mathbf{Pred}$:*

$$Not(P) \ = \ \neg P$$

Definition 44 (Conjunction and disjunction)**.** *For a finite set of predicates $C \subseteq \mathbf{Pred}$:*

$$AllOf(C) \ = \ \bigwedge_{P \in C} P \qquad AnyOf(C) \ = \ \bigvee_{P \in C} P$$

Definition 45 (Implication)**.** *For predicates* $A, B \in$ **Pred***:*

$$\mathit{Implies}(A,\, B) \;=\; A \Rightarrow B$$

where $\Rightarrow$ *is the standard logical implication.*

Definition 46 (Quantifiers)**.** *For a finite set* S *and a predicate* $P(x)$ *parameterised by* $x \in S$*:*

$$\mathit{ForAll}(S,\, P) \;=\; \bigwedge_{x \in S} P(x) \qquad \mathit{Exists}(S,\, P) \;=\; \bigvee_{x \in S} P(x)$$

Definition 47 (Event happening)**.** *For an entity* $e \in E$ *with* $t_e = \mathit{Event}$ *and a time point* $t \in T$*:*

$$\mathit{Happening}(e,\, t) \;\Leftrightarrow\; \mathit{Val}_\rho(e,\, t) = \mathit{true}$$

Definition 48 (Event has happened)**.** *For an entity* $e \in E$ *with* $t_e = \mathit{Event}$ *and a time point* $t \in T$*:*

$$\mathit{HasHappened}_\rho(e,\, t) \;\Leftrightarrow\; \exists t' \leq t :\; \mathit{Happening}(e,\, t')$$

The following predicates operate on intervals and durations. Since $\Delta \subseteq \mathbb{R}$, duration comparison is a special case of *Cmp* (Definition 41).

Definition 49 (Interval inclusion)**.** *Interval* i_1 *includes* i_2 *iff* i_2 *is entirely contained within* i_1*:*

$$\mathit{IntervalIncludes}(i_1,\, i_2) \;\Leftrightarrow\; \mathit{Start}(i_1) \leq \mathit{Start}(i_2) \;\wedge\; \mathit{End}(i_2) \leq \mathit{End}(i_1)$$

Definition 50 (Interval exclusion)**.** *Intervals* i_1 *and* i_2 *are exclusive iff they are completely disjoint:*

$$\mathit{IntervalExcludes}(i_1,\, i_2) \;\Leftrightarrow\; \mathit{End}(i_1) < \mathit{Start}(i_2) \;\vee\; \mathit{End}(i_2) < \mathit{Start}(i_1)$$

Definition 51 (Start included)**.** *The start of* i_1 *falls within* i_2*:*

$$\mathit{StartOfFirstIntervalIn}(i_1,\, i_2) \;\Leftrightarrow\; \mathit{Start}(i_2) \leq \mathit{Start}(i_1) \leq \mathit{End}(i_2)$$

Definition 52 (End included)**.** *The end of* i_1 *falls within* i_2*:*

$$\mathit{EndOfFirstIntervalIn}(i_1,\, i_2) \;\Leftrightarrow\; \mathit{Start}(i_2) \leq \mathit{End}(i_1) \leq \mathit{End}(i_2)$$

6 Operations

An operation is an action that the system performs at a specific point in time. Each operation has a set of arguments and produces *effect predicates* — a formal statement, parameterised by the firing time t, of what must hold in the trace as a result of the operation being performed. The operation is applied to some entity, which means that the argument list of every operation must include the entity the operation is applied to.

Definition 53 (Operation)**.** *An* operation *is a function*

$$\mathit{op} : T \times E \times A_1 \times \cdots \times A_n \to \mathbf{Pred}$$

for some $n \geq 0$ *and additional argument types* $A_i \subseteq V \cup E$*, where* T *is the time set and* E *is the mandatory target entity. The predicate* $\mathit{op}(t,\, e,\, a_1,\, \ldots,\, a_n)$ *is the* effect *of op applied to entity* e *at time* t *– it describes the condition that the operation causes to hold. The trigger condition that invokes an operation is specified at the requirement level. We write* O *for the finite set of all operations.*

Definition 54 (Entity type operations function)**.**

$$O_T : T_E \to \mathcal{P}(O)$$

maps each entity type and set of modifiers to its applicable operations.

The concrete mapping, including argument types and effects, is as follows. In effect predicates, t_{next} denotes the next time *any* value-modifying operation fires on entity e after t, or ∞ if no such firing occurs:

$$t_{next} = \begin{cases} \min\{\, t' \in T \mid t' > t \;\wedge\; \text{a value-modifying } op' \in O_T(t_e) \text{ fires on } e \text{ at } t' \,\} & \text{if such } t' \text{ exists} \\ \infty & \text{otherwise} \end{cases}$$

, where t is the time of the current firing and value-modifying operations are those whose effect predicate contains a $Val_\rho(e, -)$ term (i.e. *calculate*, *write*, *set_state*, *transmit*, *insert*, *remove*, *clear*, and the destination side of *read* and *receive*).

In the effect column, $EvtPred(e, ek, t)$ is used as shorthand for:

$$\forall\, ev \in E : \big(t_{ev} = \text{Event} \;\wedge\; \mu_{ev}(target) = id_e \;\wedge\; \mu_{ev}(type) = ek\big) \;\Rightarrow\; Val_\rho(ev, t) = true$$

i.e. every `Event` entity declared with the matching *target* and *type* modifiers has its value set to *true* at t, which by definition causes $Happening(ev, t)$ to hold.

Operation	**Notes**	**Effect predicates**
$calculate(t, e, expr)$	$t \in T$, $e \in E,\ t_e = \text{Signal}$, $expr \in \textbf{Expr}$	$EvtPred(e, calculated, t)$ $\forall t' \in [t, t_{next}) :\ Val_\rho(e, t') = expr(\rho, t)$
$write(t, e, expr)$	$t \in T$, $e \in E,\ t_e = \text{Storage}$, $expr \in \textbf{Expr}$	$EvtPred(e, written, t)$ $\forall t' \in [t, t_{next}) :\ Val_\rho(e, t') = expr(\rho, t)$
$read(t, e_{src}, e_{dst})$	$t \in T$, $e_{src} \in E,\ t_{e_{src}} = \text{Storage}$, $e_{dst} \in E,\ t_{e_{dst}} = \text{Signal}$	$EvtPred(e_{src}, read, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e_{dst}, t') = Val_\rho(e_{src}, t)$
$fire(t, e)$	$t \in T,\ e \in E,\ t_e = \text{EventTrigger}$	$EvtPred(e, generic, t)$
$set\_state(t, e, s)$	$t \in T$, $e \in E,\ t_e = \text{State}$, $s \in V$	$EvtPred(e, entered, t)$ $\forall t' \in [t, t_{next}) :\ Val_\rho(e, t') = s$
$transmit(t, e, expr)$	$t \in T$, $e \in E,\ t_e = \text{Channel}$, $expr \in \textbf{Expr}$	$EvtPred(e, transmitted, t)$ $\forall t' \in [t, t_{next}) :\ Val_\rho(e, t') = expr(\rho, t)$
$receive(t, e_{src}, e_{dst})$	$t \in T$, $e_{src} \in E,\ t_{e_{src}} = \text{Channel}$, $e_{dst} \in E,\ t_{e_{dst}} = \text{Signal}$	$EvtPred(e_{src}, received, t)$ $Val_\rho(e_{dst}, t) = Val_\rho(e_{src}, t)$
$insert(t, e, v)$	$t \in T$, $e \in E,\ t_e = \text{Set}$, $v \in V_{\text{nonset}}$	$EvtPred(e, inserted, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e, t') = ValBefore_\rho(e, t) \cup \{v\}$
$remove(t, e, v)$	$t \in T$, $e \in E,\ t_e = \text{Set}$, $v \in V_{\text{nonset}}$	$EvtPred(e, removed, t)$ $\forall t' \in [t, t_{next}) : Val_\rho(e, t') = ValBefore_\rho(e, t) \setminus \{v\}$
$clear(t, e)$	$t \in T$, $e \in E,\ t_e = \text{Set}$	$EvtPred(e, cleared, t)$ $\forall t' \in [t, t_{next}) :\ Val_\rho(e, t') = \emptyset$

7 Temporal constructs

So far, predicates ($\psi \in$ **Pred**) are point-in-time: they say whether something holds at a specific time t in a specific trace ρ. A *trace predicate* lifts this to the whole trace: it says whether something holds across the entire execution, not just at one moment. Formally, a trace predicate is a function $P : \Sigma^T \to \mathbb{B}$ — it takes a complete trace and returns true or false. A trace ρ *satisfies* a requirement R (written $\rho \models R$) iff R's constraint, expressed as a trace predicate P, gives $P(\rho) = \mathit{true}$.

The temporal constructs below build trace predicates from point-in-time predicates $\psi \in$ **Pred** by quantifying over time — expressing things like "ψ must hold at all times" or "whenever ψ_{cond} holds, ψ_{effect} must eventually follow."

Definition 55 (Trace predicate)**.** *A* trace predicate *is a function $P : \Sigma^T \to \mathbb{B}$. We write* **TracePred** *for the set of all trace predicates. A trace ρ* satisfies *a trace predicate P (written $\rho \models P$) iff $P(\rho) = \mathit{true}$.*

Definition 56 (Temporal constructs)**.**

$$
\begin{aligned}
\mathit{Always}(\psi) &\Leftrightarrow \forall t \in T :\ \psi(\rho, t) \\
\mathit{Eventually}(\psi) &\Leftrightarrow \exists t \in T :\ \psi(\rho, t) \\
\mathit{Initial}(\psi) &\Leftrightarrow \psi(\rho, 0) \\
\mathit{Causes}(\psi_{cond},\, \psi_{effect}) &\Leftrightarrow \forall t \in T :\ \psi_{cond}(\rho, t) \Rightarrow \exists t' \geq t :\ \psi_{effect}(\rho, t') \\
\mathit{CausesWithin}(\psi_{cond},\, \psi_{effect},\, d) &\Leftrightarrow \forall t \in T :\ \psi_{cond}(\rho, t) \Rightarrow \exists t' \in [t,\, t + d] :\ \psi_{effect}(\rho, t') \\
\mathit{Sequence}(\psi_1,\, \ldots,\, \psi_n) &\Leftrightarrow \exists t_1 \leq t_2 \leq \cdots \leq t_n \in T :\ \psi_i(\rho,\, t_i) \text{ for all } i \\
\mathit{Precedes}(\psi_1,\, \psi_2) &\Leftrightarrow \forall t_1 \in T :\ \psi_2(\rho,\, t_1) \Rightarrow \exists t_2 \leq t_1 :\ \psi_1(\rho,\, t_2) \\
\mathit{Excludes}(\psi_1,\, \psi_2) &\Leftrightarrow \forall t \in T :\ \neg\big(\psi_1(\rho,\, t) \wedge \psi_2(\rho,\, t)\big) \\
\mathit{Immediately}(\psi_1,\, \psi_2) &\Leftrightarrow \forall t_1 \in T :\ \psi_1(\rho,\, t_1) \Rightarrow \psi_2(\rho,\, \mathit{Next}(t_1))
\end{aligned}
$$

*where ψ, ψ_{cond}, ψ_{effect}, $\psi_1, \ldots, \psi_n \in$ **Pred** are point-in-time predicates and $d \in \Delta$ is a duration bound.*

The constructs compose via standard logical operators on trace predicates: conjunction $P_1 \wedge P_2$, disjunction $P_1 \vee P_2$, negation $\neg P$, implication $P_1 \Rightarrow P_2$ (if P_1 holds for trace ρ then P_2 must also hold), and biconditional $P_1 \Leftrightarrow P_2$. These operate on whole traces, not at individual time points.

8 Requirements & Test cases

Both requirements and test cases operate over the same shared entity set E, which is what makes coverage checking possible.

A requirement is a formal constraint on the system's behaviour, expressed as a trace predicate.

Definition 57 (Requirement)**.** *A* requirement *is a triple $R = (\mathit{flavour},\ \mathit{entities},\ \mathit{constraint})$ where:*

- *flavour $\in \{\mathit{Discrete}, \mathit{Continuous}\}$ fixes the time set T for this requirement (T_D or T_C respectively)*
- *entities $\subseteq E$ is the set of entities participating in this requirement*
- *constraint $\in$* **TracePred** *is the formal constraint on the trace*

A trace ρ satisfies *R (written $\rho \models R$) iff constraint$(\rho) = \mathit{true}$.*

A test case exercises a subset of the same entities to produce a concrete trace, which the coverage checker then analyses against the requirement's constraint.

Definition 58 (Stimulus)**.** *A* stimulus *is a tuple $(t,\ \mathit{op},\ e,\ a_1,\ \ldots,\ a_n)$ where:*

- $t \in T$ *is the firing time*
- *op is an operation name from the operations table*
- $e \in E$ *is the target entity*
- $a_1, \ldots, a_n \in V \cup E$ *are the additional arguments required by op*

We write **Stim** *for the set of all well-typed stimuli.*

Definition 59 (Test case)**.** *A* test case *is a triple* $\mathit{TestCase} = (\mathit{setup}, \mathit{stimuli}, \mathit{assertions})$ *where:*

- *setup is a valuation* $\sigma_0 : E \rightharpoonup V$ *establishing the initial state of the relevant entities*
- $\mathit{stimuli} \in \mathcal{P}_{\text{fin}}(\mathbf{Stim})$ *is a finite set of stimuli*
- $\mathit{assertions} : T \to \mathcal{P}(\mathbf{Pred})$ *maps each time point to the set of point-in-time predicates the test verifies at that time; it has finite support*